

Producer-Consumer Problem

Synchronization via Semaphores (Bounded-Buffer)

The Producer-Consumer problem illustrates the need for thread synchronization. Producers create data and put it into a shared buffer, while Consumers remove and process it. Semaphores prevent race conditions and handle blocking when the buffer is full or empty.

1. System Initialization

To synchronize access safely, the system requires three semaphores: a binary **mutex** (for mutual exclusion in the critical section), a counting semaphore **empty** (tracking available slots), and a counting semaphore **full** (tracking occupied slots).

```
user@os-sim:~/sync$ ./prod_cons_sim --buffer-size 3
Initializing Bounded-Buffer Simulation...
Buffer Capacity: 3 slots

# Semaphore Initialization

MUTEX = 1
(1 = Unlocked, 0 = Locked for Critical Section)

EMPTY = 3
(Initial state: 3 empty slots available)

FULL = 0
(Initial state: 0 items available to consume)

Buffer State: [ _ , _ , _ ]
```

2. Producer Thread Execution

A Producer must first decrement empty (waiting if 0), then lock the mutex. After adding the item to the buffer, it unlocks the mutex and increments full, alerting any sleeping consumers.

```
user@os-sim:~/sync$ --trigger-producer P1
```

```
[Thread-P1] Data generated: 'ITEM_A'. Attempting to insert...
```

```
[Thread-P1] wait(empty) -> SUCCESS. Decrementing EMPTY from 3 to 2.
```

```
[Thread-P1] wait(mutex) -> SUCCESS. Locked MUTEX (1 -> 0).
```

```
[Thread-P1] *** ENTERING CRITICAL SECTION ***
```

```
[Thread-P1] Inserting 'ITEM_A' into buffer[0].
```

```
[Thread-P1] Buffer State: [ ITEM_A , _ , _ ]
```

```
[Thread-P1] *** EXITING CRITICAL SECTION ***
```

```
[Thread-P1] signal(mutex) -> SUCCESS. Unlocked MUTEX (0 -> 1).
```

```
[Thread-P1] signal(full) -> SUCCESS. Incremented FULL (0 -> 1).
```

```
[Thread-P1] Operation Complete.
```

3. Consumer Thread Execution

A Consumer must first decrement full (waiting if 0, meaning buffer is empty). It then locks the mutex, extracts the item, unlocks the mutex, and increments empty.

```
user@os-sim:~/sync$ --trigger-consumer C1
[Thread-C1] Attempting to consume data...

[Thread-C1] wait(full)    -> SUCCESS. Decrementing FULL from 1 to 0.
[Thread-C1] wait(mutex)  -> SUCCESS. Locked MUTEX (1 -> 0).

[Thread-C1] *** ENTERING CRITICAL SECTION ***
[Thread-C1] Removing 'ITEM_A' from buffer[0].
[Thread-C1] Buffer State: [ _ , _ , _ ]
[Thread-C1] *** EXITING CRITICAL SECTION ***

[Thread-C1] signal(mutex) -> SUCCESS. Unlocked MUTEX (0 -> 1).
[Thread-C1] signal(empty) -> SUCCESS. Incremented EMPTY (2 -> 3).
[Thread-C1] Data 'ITEM_A' consumed successfully.
```

4. Simulation: Blocking on a Full Buffer

If producers outpace consumers and the buffer fills up, empty reaches 0. The next producer to call `wait(empty)` will be suspended by the OS until a consumer frees a slot.

```
# Fast-forwarding system state...
```

```
Current State: EMPTY = 0, FULL = 3, MUTEX = 1
```

```
Buffer State: [ ITEM_X , ITEM_Y , ITEM_Z ]
```

```
user@os-sim:~/sync$ --trigger-producer P2
```

```
[Thread-P2] Data generated: 'ITEM_W'. Attempting to insert...
```

```
[Thread-P2] wait(empty) -> BLOCKED! EMPTY is 0. Thread suspended by Kernel.
```

```
user@os-sim:~/sync$ --trigger-consumer C2
```

```
[Thread-C2] wait(full) -> SUCCESS. Decrementated FULL from 3 to 2.
```

```
[Thread-C2] wait(mutex) -> SUCCESS. Locked MUTEX (1 -> 0).
```

```
[Thread-C2] *** CRITICAL SECTION *** Removed 'ITEM_X'. Buffer: [ _ ,  
ITEM_Y , ITEM_Z ]
```

```
[Thread-C2] signal(mutex) -> SUCCESS.
```

```
[Thread-C2] signal(empty) -> SUCCESS. Incremented EMPTY (0 -> 1).
```

```
[Kernel] Waking up suspended thread [Thread-P2]...
```

```
[Thread-P2] wait(empty) -> RESUMED & SUCCESS. Decrementated EMPTY from 1 to 0.
```

```
[Thread-P2] wait(mutex) -> SUCCESS. Locked MUTEX.
```

```
[Thread-P2] *** CRITICAL SECTION *** Inserted 'ITEM_W'. Buffer: [ ITEM_W ,  
ITEM_Y , ITEM_Z ]
```